

Competitive Programming Codebook

Your Name

1 Basic Template

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);

    return 0;
}
```

2 Data Structures

2.1 Union-Find

```
struct DSU {
    vector<int> p;
    DSU(int n): p(n, -1) {}
    int find(int x){
        return p[x] < 0 ? x : p[x]
            = find(p[x]);
    }
    void unite(int a, int b){
        a = find(a); b = find(b);
        if(a == b) return;
        if(p[a] > p[b]) swap(a,b);
        p[a] += p[b];
        p[b] = a;
    }
};
```

3 Graph

3.1 Dijkstra

```
vector<long long> dijkstra(int n,
    vector<vector<pair<int,int>>> &g, int s){
    vector<long long> dist(n, 1e18);
    priority_queue<pair<long long, int>, vector<pair<long long, int>>, greater<>> pq;

    dist[s] = 0;
    pq.push({0, s});

    while(!pq.empty()){
        auto [d, u] = pq.top();
        pq.pop();
        if(d > dist[u]) continue;

        for(auto [v, w] : g[u]){
            if(dist[v] > d + w){
                dist[v] = d + w;
                pq.push({dist[v], v});
            }
        }
    }
    return dist;
}

long long modpow(long long a,
    long long b, long long mod){
    long long res = 1;
    while(b){
        if(b & 1) res = res * a % mod;
        a = a * a % mod;
        b >>= 1;
    }
    return res;
}
```

4 Math

4.1 Fast Power

5 String

5.1 KMP

```
vector<int> build_lps(string s){
    int n = s.size();
    vector<int> lps(n);
    for(int i=1, j=0; i<n; i++){
        while(j>0 && s[i]!=s[j])
            j=lps[j-1];
        if(s[i]==s[j]) j++;
        lps[i]=j;
    }
    return lps;
}
```

6 Geometry

6.1 Cross Product

```
struct Point {
    double x, y;
};

double cross(Point a, Point b){
    return a.x * b.y - a.y * b.x;
}
```

7 Notes

- long long overflow
- Graph edge case
-